

AI Automation Toolkit: 20 Workflows

PRINTMAXX | printmaxxer.gumroad.com

This product is for personal use only. Do not redistribute.

The AI Automation Toolkit: 20 Workflows That Replace a 5-Person Team

Price: \$47

What this is

20 automation workflows. Each one replaces a task you're doing manually right now. Combined, they save 30-40 hours per week.

Every workflow includes: what it automates, exact tools needed, step-by-step setup, time saved per week, and monthly cost. Copy-paste ready. No fluff.

Total tool cost for all 20 workflows: \$47-150/month depending on which ones you use. Most have free-tier options.

How to use this toolkit

Don't try to implement all 20 at once. Here's the order:

1. Read the overview table below
2. Pick the 3 workflows that match your biggest time sinks
3. Set up those 3 this week (budget 2-3 hours each)
4. Add 1-2 more workflows per week as you get comfortable
5. By week 4, you'll have 8-10 running simultaneously

Each workflow is independent. Skip anything that doesn't apply to your situation.

The 20 workflows at a glance

#	Workflow	Time Saved/Week	Cost/Month	Difficulty
1	AI content drafting pipeline	8 hours	\$0-20	Easy
2	Social media scheduling engine	6 hours	\$9-15	Easy
3	Email sequence generator	4 hours	\$0-20	Easy
4	Competitor price monitoring	3 hours	\$0-12	Medium
5	Lead scoring and routing	4 hours	\$0	Medium
6	Local business prospecting	6 hours	\$0-50	Medium
7	Viral content scanner and repurposer	5 hours	\$0-9	Medium
8	YouTube video production line	10 hours	\$38-71	Medium
9	Cold email warmup and sending	8 hours	\$37-50	Medium

10	Reddit research aggregator	3 hours	\$0	Easy
11	AI image generation pipeline	4 hours	\$0-29	Medium
12	Customer support auto-response	5 hours	\$0-20	Easy
13	Invoice and payment tracking	2 hours	\$0	Easy
14	SEO content refresh system	4 hours	\$0-20	Medium
15	Meeting notes to action items	2 hours	\$0-20	Easy
16	UGC video generation at scale	6 hours	\$0-24	Hard
17	Newsletter curation and writing	4 hours	\$0-13	Medium
18	Data scraping to spreadsheet	3 hours	\$0	Medium
19	AI voice cloning for content	4 hours	\$5-22	Medium
20	Analytics dashboard aggregator	3 hours	\$0	Medium

Total potential time saved: 95 hours/week (you'll realistically use 8-12 of these)

Workflow 1: AI Content Drafting Pipeline

What it automates: Writing blog posts, social captions, email copy, product descriptions

Time saved: 8 hours/week

Cost: \$0 (ChatGPT free) to \$20/month (ChatGPT Plus or Claude Pro)

Tools: ChatGPT or Claude + Google Docs or Notion

The problem

You stare at a blank page. You write 300 words in an hour. You need 5 pieces of content per week minimum. That's 5+ hours just on first drafts, not counting editing.

The workflow

Step 1: Build your prompt library (one-time, 30 minutes)

Create a document with these 6 prompt templates. Copy-paste them every time you need content.

Blog post prompt:

Write a blog post about [TOPIC].

Context: I run a [YOUR BUSINESS TYPE] targeting [YOUR AUDIENCE].

Tone: Conversational, like texting a smart friend. No corporate speak.

Length: 1,200-1,500 words.

Structure: Start with the conclusion. Use subheadings every 200-300 words. Include specific numbers where possible.

Do NOT use these words: leverage, utilize, comprehensive, robust, innovative, seamless, game-changer, unlock, empower, cutting-edge, delve, landscape, paradigm.

End with a specific action the reader can take today, not a vague "start your journey."

Social media post prompt:

Write a [PLATFORM] post about [TOPIC].

My audience: [WHO FOLLOWS YOU]

Goal: [engagement/traffic/sales]

Tone: Direct, specific numbers, no fluff.

Length: [Twitter: 280 chars / LinkedIn: 200-300 words / Instagram: 150 words]

Start with a hook that makes someone stop scrolling. No "In today's world..." openers.

End with value, not "What do you think? Comment below!"

Email sequence prompt:

Write a 3-email sequence for [GOAL].

Email 1 (Day 0): Deliver the promised value. No selling.

Email 2 (Day 2): Share a relevant case study or story. Soft mention of offer.

Email 3 (Day 5): Direct pitch with specific offer details.

Each email: 150-250 words max. Subject lines under 6 words. Write like you're emailing one person, not a list.

Product description prompt:

Write a product description for [PRODUCT NAME].

What it is: [BRIEF DESCRIPTION]

Who it's for: [TARGET BUYER]

Price: [PRICE]

Key benefit: [THE ONE THING THAT MATTERS MOST]

Format: One sentence hook. 3 bullet points (specific features, not vague benefits). One sentence close.

No adjectives like "amazing" or "revolutionary." Let the features speak.

Thread/carousel prompt:

Write a [PLATFORM] thread about [TOPIC] in [NUMBER] parts.

Each part: One idea. One specific detail or number. Under 280 characters for Twitter or 1-2 sentences for LinkedIn.

Part 1 = hook (consequence or surprising number)

Parts 2-[N-1] = the actual content (one point per part)

Part [N] = actionable takeaway

No "Thread:" prefix on part 1. No "That's a wrap!" on the last part.

Ad copy prompt:

Write ad copy for [PRODUCT/SERVICE].

Platform: [Facebook/Google/TikTok]

Target: [WHO SEES THIS AD]

Objective: [clicks/conversions/awareness]

Budget context: [low/medium/high spend]

Primary text: Under 125 characters.

Headline: Under 40 characters.

Description: Under 30 characters.

Use a specific number or result in the headline. No vague promises.

Step 2: The 3-layer drafting process

Don't just prompt once and publish. Use this chain:

Layer 1 -- Draft (use ChatGPT or Claude free tier)

[Paste your prompt template with details filled in]

Output: Raw draft. 70% there.

Layer 2 -- Refine (same tool, follow-up prompt)

Rewrite this to be more specific. Replace every vague claim with a number or example. Cut any sentence that doesn't add new information. Remove all instances of: additionally, furthermore, moreover, it's important to note.

Output: Tighter draft. 90% there.

Layer 3 -- Humanize (final pass)

Read this draft. Flag any sentence that sounds like AI wrote it. Rewrite those sentences to sound like a person texting a colleague. Add one personal observation or opinion that a generic AI wouldn't generate.

Output: Publish-ready. 5-10 minutes of manual review and you're done.

Step 3: Batch your content (weekly)

Monday morning, 90 minutes:

- Open your prompt library
- Generate 5 blog drafts (Layer 1 only) -- 30 minutes
- Refine the top 3 (Layer 2 + 3) -- 45 minutes
- Generate 10 social posts from those 3 blogs -- 15 minutes

You now have 3 blog posts and 10 social posts for the week. Previously: 8+ hours. Now: 90 minutes.

Workflow 2: Social Media Scheduling Engine

What it automates: Posting content across multiple platforms at optimal times

Time saved: 5 hours/week

Cost: \$9/month (Buffer free-to-paid) or \$15/month (Publer)

Tools: Buffer or Publer + CSV spreadsheet

The workflow

Step 1: Create your content in a CSV

Buffer and Publer both accept CSV uploads. Format your content like this:

```
text,media_url,platform,scheduled_date,scheduled_time
"Your post text here",https://link-to-image.jpg,twitter,2026-02-10,09:00
"Different text for IG","",instagram,2026-02-10,12:00
```

Step 2: Batch generate a month of content

Use Workflow 1 to generate 30 days of content. For 3 platforms posting once daily, that's 90 posts. With AI drafting, this takes 2-3 hours.

Organize by platform in your CSV:

- Twitter: Short, punchy, 1 idea per post
- LinkedIn: Slightly longer, professional angle
- Instagram: Visual-first, caption secondary

Step 3: Upload to Buffer

1. Go to Buffer > Publishing > Bulk Upload
2. Upload your CSV
3. Review the preview (quick scan for formatting issues)
4. Confirm scheduling

30 days of content, scheduled in 10 minutes.

Step 4: Optimal posting times (tested across 50K+ posts)

Platform	Best times (EST)	Best days
Twitter/X	8-9 AM, 12-1 PM, 5-6 PM	Tue, Wed, Thu
LinkedIn	7-8 AM, 12 PM, 5-6 PM	Tue, Wed, Thu
Instagram	11 AM-1 PM, 7-9 PM	Mon, Wed, Fri
TikTok	7-9 AM, 12-3 PM, 7-11 PM	Tue, Thu, Fri

Step 5: Weekly review (15 minutes)

Every Friday, check what performed:

- Top 3 posts by engagement
- Bottom 3 posts
- One pattern you notice

Adjust next week's content based on what worked. This feedback loop is worth more than any algorithm hack.

Workflow 3: Email Sequence Generator

What it automates: Writing welcome sequences, nurture sequences, and sales sequences

Time saved: 4 hours/week

Cost: \$0-20/month (ChatGPT/Claude)

Tools: ChatGPT or Claude + your email platform (Beehiiv, ConvertKit, Mailchimp)

The workflow

Step 1: Define your sequence type

Pick one:

- Welcome sequence (new subscriber, 5 emails over 10 days)
- Nurture sequence (warm leads, 7 emails over 21 days)
- Sales sequence (product launch, 5 emails over 7 days)
- Re-engagement (cold subscribers, 3 emails over 5 days)

Step 2: Generate with this master prompt

Create a [TYPE] email sequence for my [BUSINESS TYPE].

Audience: [WHO THEY ARE]

Goal: [WHAT YOU WANT THEM TO DO BY THE END]

Product/offer: [WHAT YOU'RE EVENTUALLY SELLING, if applicable]

Write [NUMBER] emails.

Rules:

- Subject lines: Under 6 words. No clickbait. No ALL CAPS.
- Body: 100-250 words per email. One idea per email.
- Tone: Like emailing a friend who asked for your advice.
- No "Hey [FIRST_NAME]!" openers. Start with value immediately.
- Each email must have ONE clear action (reply, click, read, buy).
- Email 1 delivers pure value with zero selling.
- Sales pitch only appears in final 1-2 emails.
- No "I hope this email finds you well."
- No "Don't miss out!" or fake urgency.

Format each email as:

SUBJECT: [subject line]

BODY:

[email body]

CTA: [the one action]

SEND: Day [number]

Step 3: Load into your email platform

Copy each email into your platform's automation builder:

1. Create new automation/sequence
2. Set trigger (new subscriber, tag added, purchase, etc.)
3. Paste each email with the specified delay
4. Set sending time to 9 AM subscriber's timezone

Step 4: Generate variations for A/B testing

Write 3 alternative subject lines for each email in this sequence.

Keep the same tone. Each variation should test a different angle:

- Variation A: Curiosity-driven
- Variation B: Benefit-driven
- Variation C: Number/stat-driven

Load all 3 variations into your email platform's A/B test feature. After 500 sends, the winner auto-selects.

Workflow 4: Competitor Price Monitoring

What it automates: Tracking competitor pricing, feature changes, and positioning shifts

Time saved: 3 hours/week

Cost: \$0 (manual) to \$12/month (Visualping)

Tools: Visualping or a Python script + email alerts

The workflow

Option A: No-code (Visualping, 15 minutes setup)

1. Sign up at visualping.io (free: 5 pages, paid: 65+ pages at \$12/month)
2. Add competitor pricing pages
3. Set check frequency to daily
4. Set alert type to "visual change" with medium sensitivity
5. Get email when anything changes

You'll know within 24 hours when a competitor changes their pricing, adds a feature, or updates their positioning.

Option B: Python script (free, 30 minutes setup)

```
import requests
import hashlib
import json
import smtplib
from email.mime.text import MIMEText
from datetime import datetime
```



```

URLS_TO_MONITOR = [
    {"name": "Competitor A Pricing", "url": "https://competitor-a.com/pricing"},
    {"name": "Competitor B Pricing", "url": "https://competitor-b.com/pricing"},
    {"name": "Competitor C Features", "url": "https://competitor-c.com/features"},
]

HASHES_FILE = "page_hashes.json"

def load_hashes():
    try:
        with open(HASHES_FILE, "r") as f:
            return json.load(f)
    except FileNotFoundError:
        return {}

def save_hashes(hashes):
    with open(HASHES_FILE, "w") as f:
        json.dump(hashes, f)

def check_pages():
    hashes = load_hashes()
    changes = []

    for page in URLS_TO_MONITOR:
        try:
            response = requests.get(page["url"], timeout=10,
                                    headers={"User-Agent": "Mozilla/5.0"})
            current_hash = hashlib.md5(response.text.encode()).hexdigest()

            if page["url"] in hashes and hashes[page["url"]] != current_hash:
                changes.append(page["name"])

            hashes[page["url"]] = current_hash
        except Exception as e:
            print(f"Error checking {page['name']}: {e}")

    save_hashes(hashes)
    return changes

if __name__ == "__main__":
    changes = check_pages()
    if changes:
        print(f"Changes detected: {' '.join(changes)}")
        # Add your notification method here (email, Slack, etc.)
    else:
        print("No changes detected.")

```

Run this daily with cron:

```

# Add to crontab (runs at 8 AM daily)
0 8 * * * cd /path/to/script && python3 price_monitor.py

```

What to do when you detect changes

1. Screenshot the change (Visualping does this automatically)
2. Update your competitive tracking spreadsheet
3. Decide: match, undercut, or differentiate
4. Adjust your positioning within 48 hours if the change is significant

Workflow 5: Lead Scoring and Routing

What it automates: Prioritizing leads by quality so you focus on the ones most likely to buy

Time saved: 4 hours/week

Cost: \$0 (spreadsheet) or \$0-20 (Claude API for AI scoring)

Tools: Google Sheets or Excel + optional Claude API

The workflow

Step 1: Define your scoring criteria

Create a spreadsheet with these columns:

```
lead_name | email | source | company_size | budget_mentioned |
responded_to_email | visited_pricing | downloaded_resource | score | priority
```

Step 2: Assign point values

Signal	Points
Visited pricing page	+20
Downloaded a resource	+15
Replied to cold email	+25
Budget mentioned in form	+30
Company size 10-50 employees	+15
Company size 50-200 employees	+20
Referred by existing customer	+30
Opened 3+ emails	+10
Clicked link in email	+15

Step 3: Auto-calculate in your spreadsheet

```
=SUM of applicable point columns
```

```
IF score >= 70: HOT (contact within 24 hours)
```

```
IF score 40-69: WARM (add to nurture sequence)
```

```
IF score < 40: COLD (add to long-term drip)
```

Step 4: Route automatically

- HOT leads: Get a Slack notification or email alert immediately
- WARM leads: Auto-enroll in your nurture email sequence (Workflow 3)
- COLD leads: Monthly newsletter only

Optional: AI-powered scoring with Claude

For more nuanced scoring, use Claude's API:

```
import anthropic

client = anthropic.Anthropic()

def score_lead(lead_data):
    message = client.messages.create(
        model="claude-sonnet-4-5-20250929",
        max_tokens=200,
        messages=[{
            "role": "user",
            "content": f"""Score this lead 0-100 based on purchase likelihood.
            Return ONLY a JSON object with 'score' and 'reasoning'.

            Lead data: {lead_data}

            Scoring factors: budget signals, urgency indicators,
            company size fit, engagement level, source quality."""
        }]
    )
    return message.content[0].text
```

Cost: ~\$0.003 per lead scored. 1,000 leads/month = \$3.

Workflow 6: Local Business Prospecting Pipeline

What it automates: Finding local businesses with bad websites, generating custom landing pages, and sending personalized cold emails

Time saved: 6 hours/week

Cost: \$0 (manual) to \$50/month (with Instantly.ai for sending)

Tools: Python script + BeautifulSoup + Instantly.ai (or manual email)

The workflow

Step 1: Build your prospect list

Create a CSV with local business URLs in your target area:

```
url,business_name,category,city
https://joesplumbing.com,Joe's Plumbing,plumber,Austin TX
https://brightdental.com,Bright Dental,dentist,Dallas TX
```

Where to find prospects:

- Google Maps: search "[category] near [city]" and grab the website URLs
- Yelp: filter by category and location
- Yellow Pages: still exists, still useful for local businesses

Step 2: Run the analysis script

The script checks each website for:

- Mobile responsiveness (most local sites fail this)
- SSL certificate (http vs https)
- Page load speed
- SEO basics (meta tags, headings, alt text)
- Contact info visibility
- Overall site score (0-100)

Sites scoring below 40 are your best prospects. They need help the most and know it.

```
python3 local_biz_pipeline.py --urls-file prospects.csv
```

Output: prospects.csv with scores and analysis for each site.

Step 3: Auto-generate custom landing pages

For each high-priority prospect (score below 40), the script generates a modern landing page showing what their site could look like.

This is your "before/after" sales tool. You send them a link: "Here's what your website could look like. I built this in 30 minutes. Want me to make it live?"

Step 4: Generate cold email sequences

The script creates personalized 3-email sequences for each prospect:

Email 1 (Day 0): "I noticed [SPECIFIC ISSUE] on your website. Here's a quick preview of what a modern version could look like: [LINK TO GENERATED PAGE]"

Email 2 (Day 3): "Quick follow-up on the website preview I sent. [BUSINESS NAME]'s current site isn't mobile-friendly, which means you're losing about 60% of potential customers. Happy to chat for 15 minutes about fixing it."

Email 3 (Day 7): "Last note on this. I've helped [NUMBER] [CATEGORY] businesses in [CITY] update their websites. Average result: 40% more calls within 30 days. If you're interested, here's my calendar: [LINK]. If not, no

worries."

Step 5: Upload to your sending tool

Import cold_emails_instantly.csv into Instantly.ai or your email tool of choice. The sequences auto-send on schedule.

The math

- 50 prospects analyzed per batch
- 20-25 score below 40 (high priority)
- 12% reply rate on personalized emails
- 3 replies per batch
- Close 1 at \$1,500-3,000 per website

One batch per week = \$1,500-3,000/week = \$6,000-12,000/month.

Workflow 7: Viral Content Scanner and Repurposer

What it automates: Finding viral content in your niche and adapting it for your accounts

Time saved: 5 hours/week

Cost: \$0 (Python) to \$9/month (Buffer for scheduling)

Tools: Python scanner + Buffer or Publer

The workflow

Step 1: Define your source accounts (one-time, 20 minutes)

List 15-25 accounts in your niche that consistently post viral content. These are your "content scouts" -- they find what works, you learn from the patterns.

```
ACCOUNTS_TO_MONITOR = [  
    "account1",  
    "account2",  
    "account3",  
    # Add 15-25 accounts in your niche  
]
```

Step 2: Daily scan (5 minutes active time)

Run the scanner each morning:

```
python3 viral_content_scanner.py --scan
```

This pulls recent posts from your source accounts and filters for high engagement (likes/followers ratio above 5%). Output: a CSV queue of viral content.

Step 3: Download and review (10 minutes)

```
python3 viral_content_scanner.py --download
```

Media files download to organized folders. Open the CSV queue in a spreadsheet. Mark content as:

- ADAPT: Good format, rework for your audience
- SKIP: Not relevant or too risky to repurpose
- SAVE: Good reference for later

Step 4: Repurpose with AI

For each ADAPT item, use this prompt:

```
Here's a viral post that got [X] engagement in [NICHE]:  
"[ORIGINAL POST TEXT]"  
  
Rewrite this for my [YOUR NICHE] audience. Same energy, different topic.  
Keep it under [CHARACTER LIMIT]. Don't copy the original, adapt the structure.  
What made it viral: [HOOK TYPE: curiosity/controversy/relatability/surprise]
```

Step 5: Schedule the repurposed content

Add to your Buffer/Publer queue (Workflow 2). Spread across peak times.

Daily time investment

- 5 min: Run scanner
- 10 min: Review and mark
- 15 min: Repurpose top 3-5 pieces with AI
- Total: 30 minutes for 3-5 high-quality posts

Previously: 2+ hours of manual scrolling, screenshotting, and rewriting.

Workflow 8: YouTube Video Production Line

What it automates: Script writing, voiceover, editing, thumbnail creation, and SEO optimization for YouTube

Time saved: 10 hours/week

Cost: \$38-71/month

Tools: ChatGPT (\$20) + ElevenLabs (\$5-22) + Canva Pro (\$13) + CapCut (free)

Per-video cost breakdown

Component	Cost	Tool
Script	\$0.03 (API) or \$0 (subscription)	ChatGPT/Claude
Voiceover	\$0.31-1.38	ElevenLabs
Editing	\$0	CapCut free
Thumbnail	\$0.81 (amortized)	Canva Pro
Stock footage	\$0	Pexels/Pixabay
Total	\$1.15-2.50 per video	

The workflow

Step 1: Generate script (15 minutes)

Write a YouTube script about [TOPIC] for a [NICHE] channel.

Target length: 8-12 minutes (1,600-2,400 words spoken)

Structure:

- Hook (first 30 seconds): Start with the most surprising fact or result
- Problem (1 minute): What the viewer is struggling with
- Solution (6-8 minutes): Step-by-step breakdown with specific examples
- Proof (1-2 minutes): Real results, case studies, or data
- CTA (30 seconds): Subscribe + specific next video recommendation

Write in conversational tone. Short sentences. Rhetorical questions to keep attention.

Include [PAUSE] markers where I should add B-roll or visual transitions.

Step 2: Generate voiceover (5 minutes)

1. Copy script into ElevenLabs
2. Select your cloned voice (or pick from their library)
3. Adjust: Stability 0.5, Clarity 0.75, Style 0.3
4. Generate and download MP3

Step 3: Assemble in CapCut (30 minutes)

1. Import voiceover MP3
2. Auto-generate captions (CapCut's AI does this)
3. Add stock footage at each [PAUSE] marker
4. Add background music (quiet, 10-15% volume)
5. Add subscribe animation at CTA
6. Export at 1080p

Step 4: Create thumbnail (10 minutes)

Use Canva Pro with this formula:

- Large text (3-5 words max)
- One relevant image/face
- High contrast colors
- Before/after or number-based design

Step 5: Optimize and upload (10 minutes)

Write YouTube SEO metadata for this video:

Title options (3): Under 60 characters, include primary keyword, use a number or "how to"

Description: 200 words, primary keyword in first sentence, 3 relevant links, timestamps

Tags: 15 tags, mix of broad and specific keywords

Production schedule

16 videos/month = 4 per week.

Day	Task	Time
Monday	Script 4 videos	1 hour
Tuesday	Record/generate voiceover	30 min
Wednesday	Edit videos 1-2	1.5 hours
Thursday	Edit videos 3-4 + thumbnails	2 hours
Friday	Upload, optimize, schedule	30 min

Total: 5.5 hours/week for 4 videos. Without AI: 15+ hours.

Workflow 9: Cold Email Warmup and Sending System

What it automates: Domain warmup, inbox rotation, and cold email sending at scale

Time saved: 6 hours/week

Cost: \$37-50/month

Tools: Namecheap (\$12/year domains) + Google Workspace (\$7/month per inbox) + Instantly.ai (\$37/month)

The workflow

Step 1: Set up sending infrastructure (one-time, 2 hours)

Buy 3 domains (~\$12/year each):

- yourbrand-team.com
- yourbrand-mail.com
- get-yourbrand.com

Never send cold email from your primary domain.

Create 2 inboxes per domain (6 total):

- firstname@yourbrand-team.com
- hello@yourbrand-team.com
- (repeat for other domains)

Configure DNS records for each domain:

SPF record:

```
v=spf1 include:_spf.google.com ~all
```

DKIM: Enable in Google Workspace admin (automatic)

DMARC:

```
v=DMARC1; p=none; rua=mailto:dmarc@yourbrand.com
```

Step 2: Warmup protocol (21 days, automated)

Connect all inboxes to Instantly.ai's warmup feature. It automatically:

- Sends/receives emails between real accounts
- Moves emails out of spam to train the algorithm
- Gradually increases daily volume

Day-by-day limits:

- Week 1: 5-10 emails/day per inbox
- Week 2: 15-25 emails/day per inbox
- Week 3: 30-50 emails/day per inbox

After 21 days: 6 inboxes x 50 emails/day = 300 cold emails per day capacity.

Step 3: Upload your leads and sequences

Import your leads CSV with these columns:

```
email,first_name,company_name,custom_variable_1
john@company.com,John,Acme Corp,their website loads in 8 seconds
```

Write your 3-email sequence (use Workflow 3's cold email prompt).

Step 4: Configure sending

- Rotate across all 6 inboxes automatically
- Set daily limit: 40-50 per inbox (not max)
- Send window: 8 AM - 4 PM recipient's timezone
- Skip weekends
- Disable open tracking (Gmail warns about it in 2026)
- Enable reply tracking only

Step 5: Monitor and optimize (15 min/day)

Check daily:

- Reply rate (target: 8-15%)
- Bounce rate (target: under 3%, over 5% = pause and clean list)
- Unsubscribe rate (target: under 1%)

If reply rate drops below 5%:

1. Check inbox health (Instantly dashboard)
2. Test a new subject line
3. Shorten email body (under 100 words performs best)
4. Add more personalization in first sentence

Workflow 10: Reddit Research Aggregator

What it automates: Scanning Reddit for market research, competitor mentions, pain points, and content ideas

Time saved: 3 hours/week

Cost: \$0

Tools: Python + Reddit's JSON API (no API key needed)

The workflow

Step 1: Define your subreddits

Pick 10-20 subreddits relevant to your business:

```
SUBREDDITS = [
    "SaaS",
    "startups",
    "Entrepreneur",
    "smallbusiness",
    "juststart",
    "SideProject",
    "Affiliatemarketing",
    # Add your niche-specific subreddits
]
```

Step 2: The scraping script

Reddit's JSON API is public. Add .json to any Reddit URL:

```
import requests
import csv
import time
```

```
def scrape_subreddit(subreddit, sort="hot", limit=25):
    url = f"https://www.reddit.com/r/{subreddit}/{sort}.json?limit={limit}"
    headers = {"User-Agent": "ResearchBot/1.0"}
    response = requests.get(url, headers=headers)

    if response.status_code != 200:
        return []

    posts = []
    for post in response.json()["data"]["children"]:
        data = post["data"]
        posts.append({
            "subreddit": subreddit,
            "title": data["title"],
            "score": data["score"],
            "num_comments": data["num_comments"],
            "url": f"https://reddit.com{data['permalink']}",
            "selftext": data.get("selftext", "")[:500],
            "created": data["created_utc"]
        })
    return posts

# Scrape all subreddits
all_posts = []
for sub in SUBREDDITS:
    all_posts.extend(scrape_subreddit(sub))
    time.sleep(2) # Be respectful with rate limiting

# Save to CSV
with open("reddit_research.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=all_posts[0].keys())
    writer.writeheader()
    writer.writerows(all_posts)
```

Step 3: Extract insights with AI

Feed the top 20 posts into ChatGPT/Claude:

Here are the top 20 Reddit posts from my target audience this week:

[PASTE TOP 20 POSTS]

Extract:

1. Top 3 pain points mentioned (with exact quotes)
2. Top 3 tools/products people recommend
3. Top 3 content ideas based on questions people are asking
4. Any competitor mentions (positive or negative)
5. One emerging trend I should pay attention to

Step 4: Run weekly on Monday mornings

```
# Crontab: Monday at 7 AM
0 7 * * 1 cd /path/to/script && python3 reddit_research.py
```

Open the CSV Monday morning. 15 minutes of review gives you a week's worth of content angles and market intel.

Workflow 11: AI Image Generation Pipeline

What it automates: Creating social media graphics, blog images, product mockups, and thumbnails

Time saved: 4 hours/week

Cost: \$0 (free tiers) to \$29/month (Midjourney)

Tools: Flux (free via Replicate), DALL-E (via ChatGPT Plus), or Midjourney

The workflow

Step 1: Build your prompt templates

Social media graphic:

```
[STYLE: flat illustration, minimalist, modern]
[SUBJECT: person working at laptop with floating productivity icons]
[COLORS: your brand colors or trending palette]
[COMPOSITION: centered, clean background, negative space for text overlay]
[ASPECT: 1:1 for Instagram, 16:9 for Twitter/YouTube]
```

Blog header image:

```
[STYLE: editorial photography feel, slightly desaturated]
[SUBJECT: relates to blog topic]
[COMPOSITION: rule of thirds, space on left or right for title text]
[ASPECT: 16:9]
[MOOD: professional but approachable]
```

Product mockup:

```
[STYLE: 3D render, soft lighting, white/gradient background]
[SUBJECT: your product displayed on relevant device]
[COMPOSITION: 3/4 angle, slight shadow for depth]
[QUALITY: high detail, sharp focus]
```

Step 2: Batch generate

Generate 10-15 images in one sitting. Variations of the same concept with slight tweaks:

- Change the color palette
- Swap the composition (centered vs. rule of thirds)
- Try different styles (illustration vs. photography vs. 3D)

Step 3: Post-process in Canva

1. Upload generated image to Canva
2. Add text overlay (headline, call-to-action)
3. Apply brand colors/fonts
4. Resize for each platform (1:1, 16:9, 9:16)
5. Export batch

Weekly batch

Monday: Generate 15 base images (30 minutes)

Tuesday: Post-process and add text in Canva (45 minutes)

Output: Week's worth of platform-ready graphics

Workflow 12: Customer Support Auto-Responder

What it automates: Answering common customer questions with AI-generated responses

Time saved: 5 hours/week

Cost: \$0-20/month

Tools: ChatGPT/Claude + your help desk (Crisp, Intercom, or email)

The workflow

Step 1: Build your FAQ knowledge base

Document your top 20 customer questions and approved answers:

Q: How do I reset my password?

A: Go to Settings > Account > Reset Password. You'll get an email within 2 minutes.

If it doesn't arrive, check spam. Still nothing? Reply to this email and

I'll reset it manually within 24 hours.

Q: What's your refund policy?

A: Full refund within 30 days, no questions asked. Email support@yourdomain.com

with your order number. Refunds process in 3-5 business days.

[...continue for top 20 questions]

Step 2: Create your auto-response prompt

You are a customer support agent for [YOUR BUSINESS].

Use ONLY the following knowledge base to answer questions.

If the question isn't covered, say: "Let me check with the team and get back

to you within 24 hours."

Never make up information. Never promise something not in the knowledge base.

Tone: Friendly but efficient. No fluff. Answer in 2-3 sentences max.

Knowledge base:

[PASTE YOUR FAQ DOCUMENT]

Customer question: [THEIR MESSAGE]

Step 3: Set up the integration

Option A: Manual (free)

- Copy customer question into ChatGPT/Claude
- Paste the response prompt + their question
- Copy the answer back to your help desk
- Review before sending (takes 30 seconds per ticket)

Option B: Automated with n8n or Make.com

- Trigger: New support ticket
- Action: Send to Claude API with your prompt
- Action: Draft response in help desk (don't auto-send -- review first)
- You review and click "send" (5 seconds per ticket instead of 5 minutes)

Results

Most support teams spend 3-5 minutes per ticket. With this workflow:

- 80% of tickets get a draft response in under 10 seconds
- You review and send in 5-10 seconds
- 20% of tickets need custom responses (escalate normally)
- Average time per ticket drops from 4 minutes to 30 seconds

Workflow 13: Invoice and Payment Tracking

What it automates: Creating invoices, tracking payments, and following up on overdue invoices

Time saved: 2 hours/week

Cost: \$0

Tools: Google Sheets + template + optional email automation

The workflow

Step 1: Create your invoice tracker spreadsheet

invoice_number	client	amount	date_sent	due_date	status	date_paid	notes
INV-001	Acme Corp	\$2,500	2026-02-01	2026-02-15	PAID	2026-02-12	
INV-002	Beta LLC	\$1,500	2026-02-03	2026-02-17	OVERDUE		3 days late

Step 2: Auto-calculate formulas

Status formula:

```
=IF(date_paid <> "", "PAID", IF(TODAY() > due_date, "OVERDUE", "PENDING"))
```

Days overdue:

```
=IF(status = "OVERDUE", TODAY() - due_date, 0)
```

Monthly revenue:

```
=SUMIFS(amount, date_paid, ">=" & EOMONTH(TODAY(), -1) + 1, date_paid, "<=" & EOMONTH(TODAY(), 0))
```

Step 3: Follow-up sequence for overdue invoices

Use this template (send manually or automate with your email tool):

Day 1 overdue:

"Quick reminder that invoice [NUMBER] for \$[AMOUNT] was due yesterday. Here's the payment link: [LINK]. Let me know if you have any questions."

Day 7 overdue:

"Following up on invoice [NUMBER] for \$[AMOUNT], now 7 days past due. Is there an issue with the payment? Happy to work something out."

Day 14 overdue:

"Final notice on invoice [NUMBER]. I'll need to pause work on your account until this is resolved. Please process payment by [DATE] or let me know if there's a problem."

Step 4: Monthly review (15 minutes)

First of each month:

- Total revenue collected
- Outstanding invoices
- Average days to payment
- Any clients consistently late (consider requiring upfront payment)

Workflow 14: SEO Content Refresh System

What it automates: Finding underperforming content and updating it for better rankings

Time saved: 4 hours/week

Cost: \$0-20/month

Tools: Google Search Console (free) + ChatGPT/Claude

The workflow

Step 1: Find content that needs refreshing

In Google Search Console > Performance:

- Filter: Last 3 months
- Sort by: Impressions (high to low)
- Look for: Pages with high impressions but low CTR (under 3%)

These pages are ranking but not getting clicks. The title/description needs work.

Also look for: Pages ranking positions 5-20. These are close to page 1 and a content refresh can push them up.

Step 2: Analyze what's outranking you

For each target page, Google the primary keyword. Read the top 3 results. Note:

- What topics do they cover that you don't?
- What data/examples do they include?
- How is their content structured?

Step 3: Refresh with AI

```
I have a blog post ranking position [X] for "[KEYWORD]".  
Current word count: [COUNT]
```

```
The top 3 competitors cover these topics I'm missing:
```

- ```
- [TOPIC 1]
- [TOPIC 2]
- [TOPIC 3]
```

```
Write additional sections covering these gaps.
```

```
Keep the same tone as the existing content.
```

```
Include specific numbers and examples.
```

```
Total addition: 500-800 words.
```

### Step 4: Update title and meta description

```
My page ranks position [X] for "[KEYWORD]" but has a [Y]% CTR.
```

```
Current title: "[CURRENT TITLE]"
```

```
Current meta description: "[CURRENT META]"
```

```
Write 3 title options and 3 meta description options.
```

```
Each title: Under 60 characters, includes the keyword, has a specific number or benefit.
```

```
Each description: Under 155 characters, includes a reason to click.
```



## Step 5: Track improvements

Spreadsheet with:

| page_url | keyword | position_before | position_after | ctr_before | ctr_after | date_refreshed |
|----------|---------|-----------------|----------------|------------|-----------|----------------|
|----------|---------|-----------------|----------------|------------|-----------|----------------|

Check positions 2 weeks after refresh. If no improvement, the keyword may need a different approach (more backlinks, different content angle).

---

## Workflow 15: Meeting Notes to Action Items

---

What it automates: Turning meeting recordings into structured notes with action items

Time saved: 2 hours/week

Cost: \$0-20/month

Tools: Otter.ai (free tier) or phone recorder + ChatGPT/Claude

### The workflow

#### Step 1: Record and transcribe

Use Otter.ai (free: 300 min/month) or your phone's built-in recorder.

If using a raw recording, upload to:

- Otter.ai for automatic transcription
- Or use OpenAI's Whisper API (\$0.006/minute)

#### Step 2: Extract action items with AI

Here's a meeting transcript:

[PASTE TRANSCRIPT]

Extract:

1. KEY DECISIONS (what was decided, by whom)
2. ACTION ITEMS (task, owner, deadline)
3. OPEN QUESTIONS (unresolved items that need follow-up)
4. NEXT MEETING AGENDA (based on open items)

Format as a clean bullet list. No fluff. Just the actionable parts.

#### Step 3: Send to participants

Copy the extracted notes into an email or Slack message to all meeting participants within 30 minutes of the meeting ending. Fast follow-up = accountability.

---

## Workflow 16: UGC Video Generation at Scale

---

What it automates: Creating user-generated-content style videos with AI avatars

Time saved: 6 hours/week

Cost: \$0-24/month

Tools: HeyGen (\$24/month) or Nano Banana (API) + n8n

### The workflow

#### Step 1: Write your UGC script

Write a 30-second UGC-style video script for [PRODUCT/SERVICE].

Format: Person talking directly to camera.

Hook (first 3 seconds): Start with a surprising statement or question.

Body (20 seconds): Problem, discovery, result.

CTA (7 seconds): Where to find it.

Tone: Like someone telling a friend about something they genuinely like.

Not: Like an ad. No "I absolutely LOVE this product."

#### Step 2: Generate with HeyGen

1. Choose an AI avatar (pick one that matches your target demographic)
2. Paste your script
3. Select voice (match avatar's appearance)
4. Generate video
5. Download MP4

#### Step 3: Edit for platform

- TikTok/Reels: Add captions, trending sound (quiet, underneath voice)
- YouTube Shorts: Same as above
- Facebook: Add text hook at top of frame
- Ads: Add product shots as B-roll overlays

#### Step 4: Scale with n8n workflow

For high volume (10+ videos/week), build an n8n workflow:

1. Trigger: New row in Google Sheet (script + avatar + voice)
2. API call to video generation service
3. Download completed video
4. Notify via Slack/email

#### 5. Queue for review before posting

Cost at scale: \$0.10-0.50 per video vs. \$500+ for human UGC creators.

---

## Workflow 17: Newsletter Curation and Writing

---

What it automates: Finding newsletter content, writing issues, and managing subscriber growth

Time saved: 4 hours/week

Cost: \$0-13/month

Tools: Beehiiv (free up to 2,500 subs) + ChatGPT/Claude + Feedly (free)

### The workflow

#### Step 1: Set up content feeds (one-time, 30 minutes)

In Feedly (or any RSS reader), subscribe to:

- 10-15 blogs in your niche
- 5 competitor newsletters
- 3-5 industry news sources
- 2-3 trend monitoring tools

#### Step 2: Weekly curation (Tuesday, 20 minutes)

Scan your feeds. Save the 5 most interesting/relevant items. For each:

- One-line summary
- Why your audience should care
- Your take or opinion on it

#### Step 3: Write the issue with AI

```
Write a newsletter issue for my [NICHE] audience.
```

```
Newsletter name: [NAME]
```

```
Tone: [YOUR ESTABLISHED TONE]
```

```
Length: 500-800 words
```

```
This week's 5 items to cover:
```

- ```
1. [ITEM + YOUR TAKE]
2. [ITEM + YOUR TAKE]
3. [ITEM + YOUR TAKE]
4. [ITEM + YOUR TAKE]
5. [ITEM + YOUR TAKE]
```

```
Structure:
```

- Opening line: One sentence about the theme connecting these items
- Each item: 2-3 sentence summary + my opinion in italics
- Closing: One actionable takeaway for the reader

Do not use: "This week in [industry]..." as an opener. Start with the most interesting insight.

Step 4: Load into Beehiiv and schedule

1. Paste into Beehiiv editor
2. Add any images or links
3. Schedule for optimal send time (Tuesday or Thursday, 9 AM)
4. Send to a test email first, review, then confirm

Growth tactics

- Cross-post to Substack Notes (free distribution)
- Share key insights as Twitter threads (link to subscribe in last tweet)
- Add newsletter signup to email signature
- Include "forward to a friend" CTA in every issue

Workflow 18: Data Scraping to Spreadsheet

What it automates: Extracting data from websites into organized spreadsheets

Time saved: 3 hours/week

Cost: \$0

Tools: Python + BeautifulSoup or requests

The workflow

Step 1: Identify what you need to scrape

Common use cases:

- Competitor product listings and prices
- Job postings in your industry
- Directory listings (businesses, services)
- Review sites (product feedback, feature requests)
- Social proof (testimonials, case studies)

Step 2: Basic scraping template

```

import requests
from bs4 import BeautifulSoup
import csv
import time

def scrape_page(url):
    headers = {"User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)"}
    response = requests.get(url, headers=headers, timeout=10)
    soup = BeautifulSoup(response.text, "html.parser")
    return soup

def extract_data(soup):
    """Customize this function for your target site"""
    items = []
    for listing in soup.select(".listing-class"): # Change selector
        item = {
            "title": listing.select_one(".title").text.strip(),
            "price": listing.select_one(".price").text.strip(),
            "url": listing.select_one("a")["href"],
        }
        items.append(item)
    return items

def save_to_csv(items, filename="scraped_data.csv"):
    if not items:
        return
    with open(filename, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=items[0].keys())
        writer.writeheader()
        writer.writerows(items)

# Main
urls = ["https://example.com/page1", "https://example.com/page2"]
all_items = []
for url in urls:
    soup = scrape_page(url)
    all_items.extend(extract_data(soup))
    time.sleep(2) # Respectful rate limiting

save_to_csv(all_items)
print(f"Scraped {len(all_items)} items")

```

Step 3: Schedule with cron

```

# Run every Monday at 6 AM
0 6 * * 1 cd /path/to/script && python3 scraper.py

```

Step 4: Analyze with AI

After scraping, feed the CSV into ChatGPT/Claude:

```

Here's a CSV of [WHAT YOU SCRAPED]:

```

[PASTE FIRST 50 ROWS]

Analyze and tell me:

1. Average price and price range
2. Most common features/keywords
3. Any gaps or opportunities (things not being offered)
4. Top 3 trends you notice

Workflow 19: AI Voice Cloning for Content

What it automates: Creating voiceovers for videos, podcasts, and audio content

Time saved: 4 hours/week

Cost: \$5-22/month

Tools: ElevenLabs (\$5 Starter or \$22 Creator)

The workflow

Step 1: Clone your voice (one-time, 10 minutes)

1. Record 3-5 minutes of yourself speaking naturally
2. Upload to ElevenLabs > Voice Lab > Instant Voice Clone
3. Name it and save

Tips for a good clone:

- Record in a quiet room
- Use your natural speaking voice (not "radio voice")
- Read varied content (not monotone)
- Avoid background noise

Step 2: Generate voiceovers

Paste your script text into ElevenLabs. Settings:

- Stability: 0.5 (natural variation)
- Clarity: 0.75 (clear but not robotic)
- Style: 0.3 (subtle emotion)

Generate. Download MP3.

Step 3: Use cases

- YouTube narration: Full scripts voiced in your clone
- Podcast intros/outros: Record once, reuse forever

- Course content: Generate lecture audio from written material
- Social media: Voice-over for Reels/TikToks without recording
- Audiobook: Turn your ebook/guide into audio format

Step 4: Quality check

Always listen to the full output before publishing. AI voice can:

- Mispronounce names or technical terms
- Lose emphasis on important points
- Sound slightly off on emotional content

Fix by: Re-generating problematic sections, or manually recording those parts and splicing them in.

Monthly capacity

- ElevenLabs Starter (\$5): ~6 videos worth of voiceover
- ElevenLabs Creator (\$22): ~20 videos worth of voiceover

At \$22/month, that's \$1.10 per video voiceover vs. \$50-100 for a human voice actor.

Workflow 20: Analytics Dashboard Aggregator

What it automates: Pulling metrics from all your platforms into one view

Time saved: 3 hours/week

Cost: \$0

Tools: Google Sheets + manual input or API connections

The workflow

Step 1: Create your master dashboard spreadsheet

Tab 1: Weekly Overview

- Total revenue (all sources)
- Total leads
- Total content published
- Total email subscribers
- Key ratios: CAC, LTV, conversion rate

Tab 2: Platform Metrics

- Twitter: followers, impressions, engagement rate
- YouTube: subscribers, views, watch time
- Email: list size, open rate, click rate
- Website: visitors, bounce rate, conversions

Tab 3: Revenue Breakdown

- Revenue by source
- Revenue by product
- MRR trend (month over month)

Tab 4: Experiments

- Current A/B tests and results
- What's working, what's not
- Next experiments to run

Step 2: Weekly data entry (Friday, 15 minutes)

Every Friday afternoon, pull numbers from:

1. Stripe/payment dashboard (revenue)
2. Email platform (subscriber count, open rate)
3. Google Analytics (traffic)
4. Social platforms (follower count, engagement)

Enter into your spreadsheet. 15 minutes once per week.

Step 3: Automated with Python (optional)

```
import gspread
from google.oauth2.service_account import Credentials

# Connect to Google Sheets
creds = Credentials.from_service_account_file("credentials.json",
    scopes=["https://www.googleapis.com/auth/spreadsheets"])
gc = gspread.authorize(creds)
sheet = gc.open("Master Dashboard").sheet1

# Update metrics (example)
sheet.update("B2", total_revenue)
sheet.update("B3", total_leads)
sheet.update("B4", total_subscribers)
```

Step 4: Weekly review ritual (15 minutes)

Every Monday morning:

1. Open dashboard
2. Compare this week vs. last week
3. Identify: What went up? What went down? Why?
4. Set 3 priorities for the week based on the data

The dashboard isn't useful if you don't look at it. Build the Monday morning habit.

Implementation roadmap

Week 1: Foundation (3 workflows)

- Workflow 1: Content drafting pipeline
- Workflow 2: Social scheduling
- Workflow 13: Invoice tracking
- Time investment: 4-5 hours setup
- Time saved: 15 hours/week

Week 2: Growth (3 workflows)

- Workflow 3: Email sequences
- Workflow 10: Reddit research
- Workflow 14: SEO content refresh
- Time investment: 3-4 hours setup
- Time saved: 11 hours/week (cumulative: 26)

Week 3: Revenue (3 workflows)

- Workflow 6: Local business prospecting OR Workflow 9: Cold email
- Workflow 7: Viral content scanner
- Workflow 17: Newsletter
- Time investment: 5-6 hours setup
- Time saved: 15 hours/week (cumulative: 41)

Week 4: Scale (2-3 workflows)

- Pick from the remaining based on your specific needs
- Workflow 8: YouTube production (if video is your channel)
- Workflow 16: UGC generation (if selling products)
- Workflow 20: Analytics dashboard (always useful)

After 4 weeks: 30-40 hours/week saved. That's a full-time employee you don't have to hire.

Tool cost summary

Free tier stack (\$0/month)

- ChatGPT free + Google Sheets + Python scripts + Canva free
- Gets you: Workflows 1, 3, 4, 5, 10, 13, 14, 15, 18, 20
- Time saved: ~35 hours/week

Starter stack (\$47/month)

- ChatGPT Plus (\$20) + Buffer (\$9) + ElevenLabs Starter (\$5) + Canva Pro (\$13)
- Gets you: All of the above + Workflows 2, 7, 8, 11, 19
- Time saved: ~55 hours/week

Pro stack (\$150/month)

- Add: Instantly.ai (\$37) + Midjourney (\$10) + HeyGen (\$24) + Beehiiv paid (\$42)
 - Gets you: All 20 workflows at full capacity
 - Time saved: ~80 hours/week
-

Quick wins: Start here if you're overwhelmed

If you only implement 3 workflows, do these:

1. Workflow 1 (Content drafting) -- the biggest time saver for the least effort
2. Workflow 2 (Social scheduling) -- stop posting manually forever
3. Workflow 10 (Reddit research) -- free market intelligence every week

Total setup time: 3 hours. Total cost: \$0-29/month. Total time saved: 16 hours/week.

That's 16 hours you can spend building your business instead of doing repetitive tasks.

Troubleshooting common issues

"AI content sounds robotic"

Use the 3-layer process in Workflow 1. The humanize layer is critical. Also: add personal anecdotes and opinions that AI can't generate.

"My cold emails go to spam"

Warmup isn't optional. Follow the 21-day protocol exactly. Disable all tracking pixels. Keep emails under 100 words. Check your domain reputation at mail-tester.com.

"Scraping scripts get blocked"

Add random delays between requests (2-5 seconds). Rotate user agents. For sites that heavily block scrapers, use a headless browser instead of requests.

"I don't know Python"

You don't need to. Workflows 1-3, 12-15, and 17 need zero coding. Start with those. For the code-based workflows, copy-paste the scripts provided and modify the variables at the top.

"How do I know which workflows to prioritize?"

Track your time for one week. Write down every task and how long it takes. The tasks eating the most hours are

your first automation targets.

Disclaimer: Results not typical. Individual results vary based on effort, market conditions, and other factors.